Emergence-as-Code as a Foundation for Self-Governing Reliable Systems

Anonymous Author(s)

Abstract

Local service-level objectives (SLOs) describe components, but journey reliability depends on an evolving interaction model—control flow, redundancy, and failure dependence—that local service-level indicators (SLIs) do not identify. This paper asks whether runtime evidence can maintain a versioned, explicitly uncertain hypothesis of that model well enough to support sound journey-level certification and rollout decisions as the system evolves.

We propose *evidence-reconciled journey reliability*: traces and configuration metadata propose a typed structural delta; reconciliation accepts or retains the model hypothesis; and a compiler derives a certifiable interval and binds the rollout record to its evidence, delta, and model version. The potential novelty is maintaining this missing model and correspondence, not dependence itself, trace discovery, or reliability algebra. A deterministic checkout replay demonstrates mechanism feasibility. Evidence of a shared payment failure domain invalidates an independence-based availability of 0.997477 and yields $[0.992514, 0.997502]$. The 0.995 objective is therefore no longer certifiable: the result is REVIEW and a risk-averse policy blocks promotion. A controlled study supports the hypothesis only if evidence-derived models track injected structural changes and yield sound decisions against static, ablated, oracle, and direct-journey baselines.

CCS Concepts

• **Software and its engineering** → **Software reliability**; *Software system models*; • **Computer systems organization** → *Self-organizing autonomic computing*.

Keywords

service-level objectives, user journeys, runtime models, self-adaptation, microservices, uncertainty

1 Problem and Research Question

SLO-as-code makes a service's objective, error budget, and alerts reviewable in the same workflow as software [2]. It does not follow that a user journey is equally well governed. A checkout request may require a front end and two parallel lookups, then succeed through either of two payment providers. Its reliability depends not only on five local SLIs, but also on which calls are required, which alternatives are active, and whether apparently redundant alternatives share a provider, zone, quota, rollout unit, or other common cause.

Journey-centric SLO practice predates this work, and OpenSLO has publicly proposed aggregating several objectives into one roll-up for an end-to-end journey [11]. Such aggregation is useful, but its inputs and weights do not reconstruct the effective control flow or the dependence among components. Likewise, SLO-allocation methods assume a microservice graph on which an end-to-end budget is distributed [10]. The graph and its failure assumptions can nevertheless change while every local SLI remains within target.

EmaC spans journey availability, latency, model discovery, and governance. This short tests one hypothesis: runtime evidence can maintain a versioned, explicitly uncertain interaction model well enough for sound journey-level certification when local SLIs do not identify system changes. Latency and the wider governance catalog are outside this claim.

Runtime models already support reasoning about changing managed systems [1, 7]; continuous assurance updates quality models [6], and feedback-loop approaches propagate uncertainty into adaptation [15, 17]. The narrower gap is the lifecycle of a decision-relevant *interaction assumption*: its changing evidence, accepted version, effect under unchanged local marginals, and consuming journey-SLO gate. This yields three necessary links; success at only one does not support the full claim:

> **RQ1—Delta validity.** Can heterogeneous evidence maintain a model hypothesis that recovers an injected structural change and its field-level provenance at useful review coverage?
> **RQ2—Predictive fidelity.** Do reconciled intervals improve coverage, width, and false-certification risk over frozen interaction assumptions?
> **RQ3—Decision utility.** Does reconciliation reduce harmful promotions at comparable false-block and review budgets?

The information gap is formal, not merely a missing dashboard. Let S_a and S_b denote success of two alternatives with local marginals A_a and A_b. Without a dependence model, their race success is only constrained by the Fréchet bounds

$$\max(A_a, A_b) \le \Pr[S_a \cup S_b] \le \min(1, A_a + A_b). \tag{1}$$

At $A_a = A_b = 0.995$, both an independent race with availability 0.999975 and perfectly shared fate with availability 0.995 are compatible with the same local SLIs. A journey-level declaration supplies the control-flow operator, but it still cannot choose between these values unless its interaction assumptions are maintained. The research object is therefore not "an SLO over several services"; it is the evidence-to-assumption link that keeps such an SLO decision-relevant as the system changes.

We call the mechanism **Emergence-as-Code** (EmaC). *Emergent* means only that local marginals do not identify journey reliability; this is motivation, not novelty. The contribution hypothesis is whether the missing interaction model can be maintained as a versioned, uncertain runtime hypothesis and safely consumed by certification. We provide its minimal delta contract, a mechanism replay, and an end-to-end evaluation plan. This short studies availability; latency under retries, queueing, and feedback remains a separate extension.

Table 1: Minimum reconciliation contract for one model delta.

Field	Required content and purpose
Changed claim	Typed path plus before/after values for an edge, operator, or effective failure domain
Evidence slice	Time window, source identifiers, coverage/agreement, and field-level provenance
Predicted impact	Journey interval before/after and certification consequence
Model disposition	Proposed, accepted, or retained-old-model, with invariant and evidence checks
Decision record	PASS/FAIL/REVIEW plus PROMOTE/BLOCK, exact reasons, and source model/delta versions

2 Evidence-Reconciled Journey Reliability

2.1 One model delta, end to end

Let J be declared journey intent, $\mathbf{s}_t$ the vector of local availability estimates, and M_t the accepted interaction-model hypothesis at time t. It contains only decision-relevant structure: the typed control-flow expression and effective failure-domain assignments. Runtime evidence E_t does not turn it into ground truth or silently overwrite it. Instead, reconciliation proposes a typed delta Δ_t and an evidence-support score c_t:

$$\begin{aligned}(\Delta_t, c_t) &:= \text{propose}(E_t, M_{t-1}),\\ \widetilde{M}_t &:= \text{apply}(M_{t-1}, \Delta_t),\\ \widetilde{I}_t &:= [\widetilde{R}_t^-, \widetilde{R}_t^+] := \text{compile}(J, \mathbf{s}_t, \widetilde{M}_t),\\ (q_t, a_t, M_t) &:= \text{reconcile}(M_{t-1}, \widetilde{M}_t, \widetilde{I}_t, c_t, P).\end{aligned} \tag{2}$$

$\widetilde{M}_t$ and $\widetilde{I}_t$ are candidates, not silently accepted state. P separately governs model disposition, epistemic certification q_t, and operational action a_t. Reconciliation accepts $\widetilde{M}_t$ as M_t only when its typed invariants and evidence policy hold; otherwise $M_t = M_{t-1}$ while the candidate analysis remains inspectable. For objective $R \geq \theta$, certification is

$$q_t = \begin{cases} \textsc{pass}, & \widetilde{R}_t^- \geq \theta,\\ \textsc{fail}, & \widetilde{R}_t^+ < \theta,\\ \textsc{review}, & \widetilde{R}_t^- < \theta \leq \widetilde{R}_t^+. \end{cases} \tag{3}$$

A risk-averse policy may map REVIEW to BLOCK, but it may not relabel uncertainty as an observed SLO violation.

A delta records the changed field, old and candidate values, evidence window, provenance, and predicted effect on $\widetilde{R}_t$ (Table 1). The score c_t summarizes coverage and agreement; it is not a calibrated probability. Missing or conflicting evidence routes to REVIEW, consistent with explicit uncertainty management [8]. Model Discovery is replaceable: existing dependency and causal-latency methods may supply evidence [9, 18], but their algorithms are not claimed as contributions.

Before acceptance, reconciliation checks typing, field evidence and temporal scope, the exact prior version, conflicts, and the PASS thresholds. Records are deterministic in the accepted version, inputs, and policy. Weak candidates may be compiled conservatively but cannot advance M_t automatically.

Model acceptance and rollout remain distinct. A well-supported shared-domain delta may be accepted while its interval straddles 0.995, yielding REVIEW and risk-averse BLOCK. Weak evidence instead retains the old model and also yields REVIEW. Neither c_t nor a high bound compensates for failure of the other; FAIL requires an upper bound below the objective.

The running journey is intentionally small:

```
Series(front, Parallel(cart, pricing),
  Race(payA, payB)),   A_J ≥ 0.995.
```

Suppose the two payment alternatives have availability A_a and A_b. If the accepted model supports independent effective failure domains, the replay uses the explicit independence estimate $A_{\text{race}}^{\text{ind}} = 1-(1-A_a)(1-A_b)$. If evidence instead shows a possible common failure domain, it invalidates independence but does not prove perfect shared fate. The defensible race result is therefore the Fréchet interval $[\max(A_a, A_b), \min(1, A_a + A_b)]$. The replay fixes independence between the non-payment stages and varies only this payment assumption; those unchanged cross-stage assumptions are recorded as scope, not treated as universal truth. Other stages and $\mathbf{s}_t$ are unchanged. Consequently,

$$\mathbf{s}_{t-1} = \mathbf{s}_t \quad \Rightarrow \quad R(J, \mathbf{s}_{t-1}, M_{t-1}) = R(J, \mathbf{s}_t, M_t). \tag{4}$$

This is the paper's central causal story. Runtime evidence invalidates or changes an interaction hypothesis; if accepted, the revised hypothesis changes a dependence-sensitive journey interval; reconciliation certifies, rejects, or abstains; and provenance makes both the model transition and the action testable and reproducible.

Table 2: Closest mechanisms and the differential claim.

Work	What it already provides	Differential tested here
KAMI [5]	Runtime parameter updates and requirement prediction	Structural, field-provenanced interaction deltas tied to certification
QoSMOS [4]	Runtime probabilistic QoS analysis and adaptation for service compositions	Accepted model version and journey-bound rollout record
EUREMA [16]	Executable runtime megamodels and feedback loops	Evidence-to-dependence-delta contract with SLO semantics
Runtime provenance [13]	Provenance graphs and runtime-model history	Reliability meaning of a delta and deterministic gate consumption
Bayesian availability [3]	Replication, cascading, and common-cause failure models	Operational proposal/reconciliation and versioned deployment decision
This work	Typed structural delta with field evidence	Delta → accepted model → interval → decision invariant

2.2 Claim boundary

Table 2 locates the proposal. Neither interaction dependence nor any row in isolation is the novelty hypothesis. The hypothesis tested across the final row is that heterogeneous evidence can maintain a versioned, explicitly uncertain interaction model whose accepted state controls certification. Versioning and provenance provide its assurance semantics: they make model-tracking error and decision soundness inspectable without treating the model as ground truth.

The proposal is an integration contribution with a testable differential; Table 2's ingredients already exist. ParSLO allocates a budget on a supplied graph [10], while workflow tracing reconstructs executions [14]. EmaC instead tests an evidence-maintained interaction hypothesis under cross-artifact invariants: evidence supports the changed field, the accepted version determines the interval, and the decision cites both.

Hence the baselines are a frozen compiler, metadata-only and traces-only ablations, an oracle delta, and a direct journey probe.

Table 3: Sanity replay. Local inputs and the independence calculation $A_J^{\text{ind}} = 0.99747706$ are fixed.

Scenario	Relation	Compiled I_J	c_t	Cert./action
Baseline	independent	[.997477,.997477]	.95	PASS/promote
Shared-domain drift	dependent	[.992514,.997502]	.95	REVIEW/block
Weak/conflicting evidence	unresolved	[.992514,.997502]	.35	REVIEW/block

Discovery methods [9, 18] may supply evidence, but their accuracy is not EmaC novelty; link-level oracles keep a lucky final action from hiding a wrong delta.

3 Preliminary Feasibility Evidence

An executable, deterministic replay exercises Equation 2 on the checkout journey. Its atomic inputs remain fixed and green: $A_{front} = 0.9995$, $A_{cart} = A_{pricing} = 0.999$, and $A_a = A_b = 0.995$. Synthetic trace spans and deployment/provider labels provide the evidence. The prototype validates accepted parent–child edges against spans, checks whether payment spans implement a race, compares domain observations with the accepted model, emits a typed delta, recompiles journey availability, and evaluates the gate; it does not infer an arbitrary graph. The replay and all generated reports are included as anonymized supplementary material.

For reproducibility, the replay's heuristic score is $c_t = .40c_{edge} + .25c_{race} + .35c_{domain}$, with a 0.85 penalty when missing/conflicting-evidence flags are present and an auto-accept threshold of 0.80. Full coverage and source agreement give the three field scores 0.95. The weights are fixtures, not learned or validated probabilities: their role is to exercise abstention and make the later calibration question measurable.

Table 3 reports the complete result. In every row, the local inputs and the independence estimate $A_J^{\text{ind}} = 0.99747706$ are identical. In the drift row, trace attributes and deployment metadata agree that `payA` and `payB` moved from separate domains into `payment-network-shared`. This evidence invalidates the independence point model. It does not identify the joint distribution, so the compiled journey result becomes $[0.99251449, 0.99750200]$ rather than the lower endpoint alone. The 0.995 objective lies inside that interval: certification changes from PASS to REVIEW, and the configured risk-averse policy blocks promotion. In the weak-evidence row, one payment branch is absent from traces and domain metadata conflict; the prototype exposes the same conservative candidate range but refuses the model update as well as automatic promotion.

This replay establishes only mechanism coherence: a model-only change can propagate to an explainable decision while local marginals remain fixed. It does *not* establish discovery accuracy, bound tightness, generality, or operational benefit. Those are precisely the claims assigned to the research plan rather than smuggled into the proof of concept.

The generated record is more than the three labels in Table 3. For every changed payment-domain field it retains the accepted value, candidate value, deployment-label and span-attribute locations, field support, old/new reliability interval, and policy reason. This gives the planned study an auditable notion of explanation correctness: provenance is correct only when it identifies evidence generated by the injected cause, not merely evidence correlated with the final decision. A pipeline can therefore be wrong even if it happens to emit the right certification or action.

4 Research Plan

The evaluation asks whether an evidence-derived model tracks injected interaction changes and whether decisions conditioned on it remain sound; Equation 2's links are necessary diagnostics. We extend the OpenTelemetry Demo checkout with two payment backends on Kubernetes [12]. Controlled changes provide structural ground truth, a canary window supplies E_t, and a disjoint post-change window measures outcomes.

The replicate is an independent run block, not a request. Each block restores the deployment, randomizes a candidate, and separates evidence and outcome windows. Confirmatory blocks isolate one transition; an ordered no-reset sequence tests version lineage, stale-evidence rejection, rollback, and replay. The partial factorial crosses failure-domain movement, four evidence conditions (complete, down-sampled, missing labels, conflicting sources), and five common-cause strengths from independence to perfect shared fate. Alternative activation and a shared external dependency are exploratory replications. Seeds are paired across methods and block order randomized within strength. A pilot estimates paired variance; blocks are then preregistered for 80% power with Holm-controlled familywise $\alpha = .05$ on two co-primary endpoint families: selective risk of accepted fused-evidence deltas versus both single-source ablations at matched auto-accept coverage, and false-certification risk versus the frozen compiler at matched REVIEW and false-block budgets over nonzero dependence strengths. Weighted interval score is a required intermediate metric, not a substitute for either endpoint. Requests improve within-block precision but are not replicates.

The causal contrast fixes local data-generating processes while intervening on dependence. A seeded provider event supplies a predeclared fraction of branch failures; branch-specific events preserve configured marginals from independent to perfectly shared fate. Expected marginals are fixed without selecting runs after observing outcomes. Request populations and windows are common; downstream SLIs are conditional success probabilities under declared call, retry, hedging, and cancellation semantics. Achieved per-block marginals and sampling intervals are balance checks. The estimand is the paired journey change due to dependence, not load or component quality.

Controls separate evidence processing from reliability: no change should yield no delta; conflicting labels under unchanged execution should increase REVIEW, not count as a correct failure; and down-sampled traces test whether metadata preserves coverage. Neither change nor missing spans implies shared fate.

Table 4 makes the claim conjunctive: a correct decision cannot mask a wrong delta, nor can an accurate delta compensate for unsound certification.

Delta validity. Against a withheld injection manifest, we measure field precision/recall, provenance correctness, delay, and selective risk as the REVIEW threshold changes. Because c_t is support rather than probability, we use risk–coverage, ranking loss, and

Table 4: Link-level predictions, oracles, and falsification signals.

Link	Oracle, prediction, and falsification signal
Propose	Against the injected manifest, accepted deltas recover changed fields at the preregistered selective-risk ceiling; falsified by persistent error or coverage achieved only through blanket REVIEW
Compile	Against held-out outcomes, intervals reduce weighted interval score and false certification; falsified if they do not beat the frozen model
Reconcile	Against injected harmful/benign labels, the Pareto frontier improves; falsified if safety requires blanket blocking or REVIEW

AUROC, fixing its threshold on pilot blocks before confirmation. The oracle is never available to discovery.

Estimate fidelity. Injectors match per-service marginals while changing dependence, so local SLIs cannot reveal the journey difference. Canary inputs generate an interval tested on subsequent outcomes. We report coverage, width, weighted interval score, false certification, and certification rate; point error only for point predictors. Paired differences use bootstrap 95% intervals over blocks. Independence is a negative control: extra width is conservative, not more point-accurate.

Decision utility. Identical seeds feed local-green and weighted-roll-up controls; frozen, single-source, fused, oracle-delta, and direct-journey comparators. The direct probe tests whether structural evidence enables an earlier or more sample-efficient decision. Injected ground truth, with a preregistered indifference zone around 0.995, defines harm; noisy empirical outcomes are a sanity check, not the label. We report false promotion, false block, REVIEW, delay, and requests-to-decision as a Pareto frontier; REVIEW is charged, never counted as correct rejection. Proposal, reconciliation, and compilation latency, telemetry/storage overhead, model churn, and human reviews are recorded. Configurations, seeds, windows, outcomes, and deltas are released.

Failure is informative. Traces may miss a shared provider or quota until an incident; if multi-source evidence cannot recover decision-relevant deltas at low selective risk, only an advisory compiler routing changes to REVIEW is supported. Workload non-stationarity may still confound windows despite paired schedules, randomized blocks, and held-out outcomes. One system, one threshold, and one journey cannot establish generality; replication across systems and control-flow operators remains necessary.

Interpretation. An oracle-only success supports a manual compiler; sound intervals without a better decision frontier show only feasibility; and blanket REVIEW is not success. The hypothesis requires low-risk structural tracking and reduced false certification at comparable REVIEW/false-block budgets, with interval fidelity linking them.

5 Conclusion

Journey reliability depends on an evolving interaction model that local SLIs do not identify. This paper asks whether runtime evidence can maintain a versioned, uncertain hypothesis of that model well enough for sound certification and rollout decisions. EmaC operationalizes evidence → delta → accepted model → interval → version-linked decision. With fixed local SLOs, the replay invalidates a point-valued PASS and yields $[0.992514, 0.997502]$, hence REVIEW/BLOCK. The study supports the claim only if accepted deltas track injected structure with low selective risk and decisions remain sound at comparable review budgets.

References

[1] Nelly Bencomo, Sebastian Götz, and Hui Song. 2019. Models@run.time: A Guided Tour of the State of the Art and Research Challenges. *Software and Systems Modeling* 18, 5 (2019), 3049–3082. doi:10.1007/s10270-018-00712-x

[2] Betsy Beyer, Niall Richard Murphy, David K. Rensin, Kent Kawahara, and Stephen Thorne (Eds.). 2018. *The Site Reliability Workbook: Practical Ways to Implement SRE.* O'Reilly Media, Sebastopol, CA, USA.

[3] Otto Bibartiu, Frank Dürr, Kurt Rothermel, Beate Ottenwälder, and Andreas Grau. 2024. Availability Analysis of Redundant and Replicated Cloud Services with Bayesian Networks. *Quality and Reliability Engineering International* 40, 1 (2024), 561–584. doi:10.1002/qre.3414

[4] Radu Calinescu, Lars Grunske, Marta Kwiatkowska, Raffaela Mirandola, and Giordano Tamburrelli. 2011. Dynamic QoS Management and Optimisation in Service-Based Systems. *IEEE Transactions on Software Engineering* 37, 3 (2011), 387–409. doi:10.1109/TSE.2010.92

[5] Ilenia Epifani, Carlo Ghezzi, Raffaela Mirandola, and Giordano Tamburrelli. 2009. Model Evolution by Run-Time Parameter Adaptation. In *Proceedings of the 31st International Conference on Software Engineering (ICSE '09)*. IEEE Computer Society, 111–121. doi:10.1109/ICSE.2009.5070513

[6] Antonio Filieri, Carlo Ghezzi, and Giordano Tamburrelli. 2012. A Formal Approach to Adaptive Software: Continuous Assurance of Non-functional Requirements. *Formal Aspects of Computing* 24, 2 (2012), 163–186. doi:10.1007/s00165-011-0207-2

[7] David Garlan, Shang-Wen Cheng, An-Cheng Huang, Bradley Schmerl, and Peter Steenkiste. 2004. Rainbow: Architecture-Based Self-Adaptation with Reusable Infrastructure. *Computer* 37, 10 (2004), 46–54. doi:10.1109/MC.2004.175

[8] Sara M. Hezavehi, Danny Weyns, Paris Avgeriou, Radu Calinescu, Raffaela Mirandola, and Diego Perez-Palacin. 2021. Uncertainty in Self-adaptive Systems: A Research Community Perspective. *ACM Transactions on Autonomous and Adaptive Systems* 15, 4 (2021), 1–36. doi:10.1145/3487921

[9] Anatoly A. Krasnovsky. 2026. Model Discovery and Graph Simulation: A Lightweight Gateway to Chaos Engineering. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER '26)* (Rio de Janeiro, Brazil). Association for Computing Machinery, New York, NY, USA, 121–125. doi:10.1145/3786582.3786823

[10] Amirhossein Mirhosseini, Sameh Elnikety, and Thomas F. Wenisch. 2021. Parslo: A Gradient Descent-based Approach for Near-optimal Partial SLO Allotment in Microservices. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*. Association for Computing Machinery, New York, NY, USA, 442–457. doi:10.1145/3472883.3486985

[11] OpenSLO Contributors. 2021. SLO Aggregation. OpenSLO GitHub issue 41. https://github.com/OpenSLO/OpenSLO/issues/41 Opened July 15, 2021; accessed August 23, 2026.

[12] OpenTelemetry Authors. 2026. OpenTelemetry Demo Documentation. OpenTelemetry documentation. https://opentelemetry.io/docs/demo/ Accessed August 23, 2026.

[13] Owen Reynolds, Antonio García-Domínguez, and Nelly Bencomo. 2020. Automated Provenance Graphs for Models@run.time. In *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. Association for Computing Machinery, Article 53, 10 pages. doi:10.1145/3417990.3419503

[14] Raja R. Sambasivan, Ilari Shafer, Jonathan Mace, Benjamin H. Sigelman, Rodrigo Fonseca, and Gregory R. Ganger. 2016. Principled Workflow-Centric Tracing of Distributed Systems. In *Proceedings of the Seventh ACM Symposium on Cloud Computing (SoCC '16)*. Association for Computing Machinery, New York, NY, USA, 401–414. doi:10.1145/2987550.2987568

[15] Gabriela Félix Solano, Ricardo Diniz Caldas, Genaína Nunes Rodrigues, Thomas Vogel, and Patrizio Pelliccione. 2019. Taming Uncertainty in the Assurance Process of Self-Adaptive Systems: A Goal-Oriented Approach. In *Proceedings of the 14th IEEE/ACM International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '19)*. IEEE, Montreal, QC, Canada, 89–99. doi:10.1109/SEAMS.2019.00020

[16] Thomas Vogel and Holger Giese. 2012. A Language for Feedback Loops in Self-Adaptive Systems: Executable Runtime Megamodels. In *Proceedings of the 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '12)*. IEEE Computer Society, 129–138. doi:10.1109/SEAMS.2012.6224399

[17] Danny Weyns and Usman M. Iftikhar. 2023. ActivFORMS: A Formally Founded Model-based Approach to Engineer Self-adaptive Systems. *ACM Transactions on Software Engineering and Methodology* 32, 1, Article 12 (2023), 48 pages. doi:10.1145/3522585

[18] Yazhuo Zhang, Rebecca Isaacs, Yao Yue, Juncheng Yang, Lei Zhang, and Ymir Vigfusson. 2023. LatenSeer: Causal Modeling of End-to-End Latency Distributions by Harnessing Distributed Tracing. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '23)*. Association for Computing Machinery, New York, NY, USA, 502–519. doi:10.1145/3620678.3624787